

Taller Express y otras cosas



Nombre:

Lucas Alexander Mejia Diaz

Matrícula:

A01646442

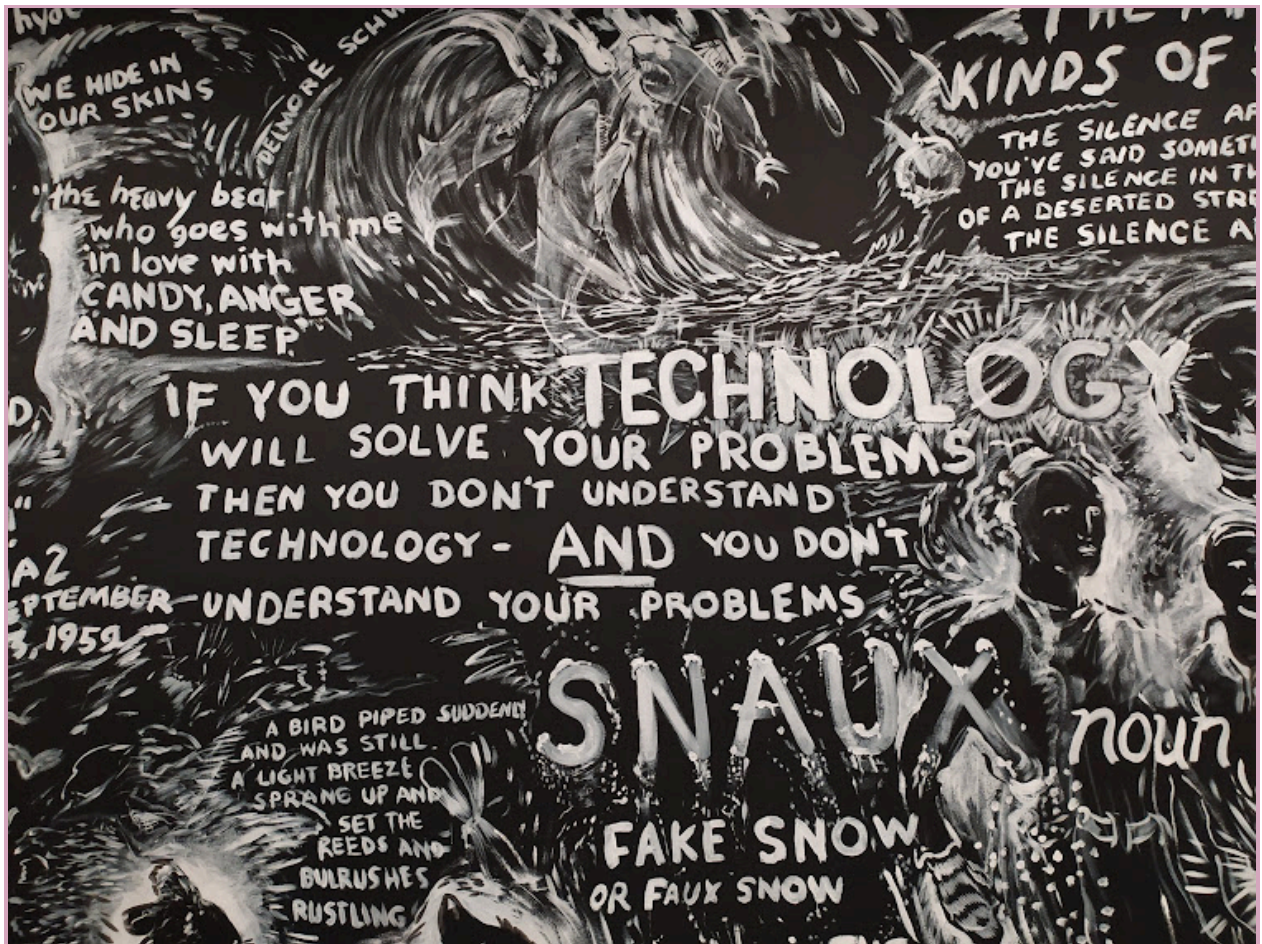
¿Quién eres?:

COMO un homosapiensapien

El objetivo de la actividad es explorar y conocer más sobre el desarrollo web del lado del backend.

Al final de la actividad deberás hacer un PDF con este documento. El PDF deberá tener un enlace en [miarchivo.html](#)

- **Ejercicio -1: Una pregunta**



¿Qué piensas de la frase? ¿Qué problema crees que podrías resolver con tecnología?

No es del todo cierta ya que no todo se puede resolver con la tecnología, la tecnología puede resolver problemas y facilitar muchas cosas y automatizarlas, pero problemas más de ética y moral no los puede resolver .

- **Ejercicio 0: El pasado**

En las semanas previas vimos la forma de consumir un API desde el cliente (el navegador). Ya consumiste el **API desde el servidor** usando javascript y nodejs.

Escribe aquí abajo el enlace al archivo (con esa solución) en tu repositorio:

[LucasMejiaDiaz.github.io/server.js at main · LucasMejiaDiaz/LucasMejiaDiaz.github.io · GitHub](https://LucasMejiaDiaz.github.io/server.js)

¿Cuál fue el principal problema para resolver este ejercicio? ¿Cuál fue el problema que tuvo una de las personas en tu equipo? Explica cómo resolvió esa persona ese problema.

El principal problema fue entender que la API no debía consumirse directamente desde el navegador, sino desde el servidor con Node.js. Para resolverlo hice una ruta /clima en Express, recibí la ciudad con POST, consulté la API de Weatherstack desde el servidor y regresé los datos al cliente en formato JSON.

• Ejercicio 1: Simplicidad

¿ Recuerdas cómo **instalar un módulo** en node.js? Escribe la instrucción que se usa:

npm install nombre-del-modulo

• Ejercicio 2: Express es

Investiga qué es express (en el contexto de desarrollo web en nodejs). Explica a un **niñ@** qué es express, cómo se usa, para qué sirve. Escribe ese texto aquí. Incluye también **las referencias** de tu investigación.

imagina que una página web es como una tienda. Cuando una persona entra y pide algo, alguien tiene que escuchar el pedido, buscar la información y responder. Express

es como el encargado de la tienda: recibe las solicitudes, revisa qué quiere la persona y manda una respuesta.

Audio explicacion: 📻 WIN_20260512_09_59_08_Pro.mp4

<https://expressjs.com/>

https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs/Introduction

Express se usa instalándolo con: `npm install express` Después se importa en JavaScript, se crea una aplicación y se agregan rutas. Por ejemplo, una ruta puede responder cuando alguien entra a `/`, otra cuando entra a `/usuarios`, y otra cuando manda datos desde un formulario.

Dile a una persona de la clase (que no esté en tu mesa) que te explique qué es express y qué ventajas te podría dar su uso. Graba el audio de la explicación.

¿Crees que realizó una buena explicación? ¿Por qué?

no , darío garay no me supo explicar de buena manera

• Ejercicio 3: Desde un libro

Investiga una definición en un libro (digital o físico) **de una API**. Incluye la referencia del libro.

Escribe un ejemplo de un endpoint de tu proyecto.

• Ejercicio 4: Instala express

Escribe la instrucción que se usa para **instalar el módulo de express**:

Según Martin Reddy, en el libro API Design for C++, una API es una forma de describir cómo un programa puede interactuar con un componente de software. Es decir, una API sirve como una interfaz que oculta los detalles internos y permite que otro programa use ciertas funciones o datos sin saber exactamente cómo están contruidos por dentro.

En palabras simples, una API es como un menú: te dice qué puedes pedir, cómo pedirlo y qué respuesta vas a recibir.

Referencia del libro

Reddy, Martin. API Design for C++. Morgan Kaufmann / Elsevier, 2011.

Referencia:

<https://www.sciencedirect.com/book/monograph/9780123850034/api-design-for-c-and-and>

Ejemplo de endpoint de mi proyecto

Un endpoint de mi proyecto es:

POST <http://localhost:3000/clima>

Ese endpoint recibe una ciudad desde el navegador, consulta la API de Weatherstack desde el servidor y devuelve datos como la ciudad, el país, la temperatura y la descripción del clima.

• Ejercicio 5: Uso de express

Ejecuta y explica qué hace el siguiente código:

```
import express from 'express';

const app = express();

app.listen(1984, () => {
  console.log('Up and up');
});
```

Ejercicio 5: Uso de Express

Este código crea un servidor básico usando Express en Node.js.

Primero se importa Express con esta línea:

```
import express from 'express';
```

Después se crea una aplicación de Express:

```
const app = express();
```

Esa variable app representa el servidor que vamos a configurar. Ahí se pueden agregar rutas, recibir solicitudes y enviar respuestas.

Finalmente, esta parte inicia el servidor:

```
app.listen(1984, () => {
  console.log('Up and up');
});
```

Esto significa que el servidor empieza a escuchar solicitudes en el puerto 1984. Cuando el servidor se inicia correctamente, muestra en la terminal el mensaje:

Up and up

• Ejercicio 6: Uso de express++

Ejecuta y explica qué hace el siguiente código (agregado al ejercicio 5):

```
app.get('/bienvenida', (req, res) => {  
  res.send('Esto no es una página html');  
});  
  
app.get('/otraBienvenida', (req, res) => {  
  res.sendFile('bienvenida.html');  
});
```

En esta parte le agregué dos rutas (o endpoints) al servidor para que sepa qué responder cuando alguien entre a direcciones específicas:

1. Ruta /bienvenida: Aquí usé `res.send`. Cuando entro a esa dirección desde el navegador, el servidor simplemente me regresa el texto: "Esto no es una página html". Es una respuesta directa de texto plano.
2. Ruta /otraBienvenida: Aquí usé `res.sendFile`. La idea es que, en lugar de un texto, el servidor busque un archivo real llamado `bienvenida.html` y lo muestre en el navegador como una página web completa.

En caso de que alguno de estos te marque error, soluciona el problema y explica la forma de solucionarlo:

Ejercicio 5 y 6: Uso de Express y Manejo de Rutas

Al ejecutar el código original, el servidor inicia correctamente y muestra el mensaje "Up and up" en la terminal. Sin embargo, existe un problema técnico con la segunda ruta.

El Error:

Al entrar a `http://localhost:1984/otraBienvenida`, el servidor arrojará un error. Esto sucede porque el método `res.sendFile()` de Express no acepta rutas relativas (como solo poner el nombre del archivo). Node.js necesita la ruta absoluta, es decir, la dirección completa desde la raíz de tu disco duro hasta el archivo.

La Solución:

Para arreglarlo, debemos importar el módulo **path** (que ya viene en Node) y configurar unas variables para que el servidor sepa exactamente en qué carpeta estamos trabajando.

CODIGO:

```
import express from 'express';
import path from 'path'; // Importamos path para manejar rutas
import { fileURLToPath } from 'url';

const app = express();

// Configuración para obtener la ruta de nuestra carpeta actual
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

app.listen(1984, () => {
  console.log('Up and up');
});

app.get('/bienvenida', (req, res) => {
  res.send('Esto no es una página html');
});

app.get('/otraBienvenida', (req, res) => {
  // Solución: usamos path.join para dar la ruta absoluta correcta
  res.sendFile(path.join(__dirname, 'bienvenida.html'));
});
```

• Ejercicio 7: Recuerdo de imagen

Es posible si pusiste una imagen en el servidor.js no se podía ver. Un servidor entrega respuestas a solicitudes realizadas por un cliente. Cuando el cliente solicita una imagen (u otro tipo de archivo) el servidor tiene que saber responder.

Express nos propone crear un directorio para los archivos que queremos usar, por ejemplo imágenes o archivos de hojas de estilo.

Investiga qué es `express.static`.

Escribe tu investigación

`express.static` es una función de Express que sirve para compartir archivos estáticos desde el servidor. Los archivos estáticos son archivos que no cambian cuando el usuario los pide, por ejemplo imágenes, archivos CSS, archivos JavaScript del cliente, fuentes o documentos. Cuando usamos un servidor con Express, el navegador no puede ver automáticamente todos los archivos de nuestro proyecto. Si una página HTML tiene una imagen como `foto1.png`, el navegador hace otra solicitud para pedir esa imagen. El servidor necesita saber desde qué carpeta puede entregar ese archivo.

Para eso sirve `express.static`.

Ejemplo:

```
app.use(express.static('public'));
```

Con esa línea le decimos a Express que la carpeta `public` tendrá archivos que el navegador puede pedir directamente.

• Ejercicio 8: Canción

Pregunta a una persona de otro equipo su canción favorita y sus razones.

Waka waka shakira, que por que le recuerda el mundial de futbol 2010

¿Ya la conocías? ¿Te gusta? ¿Por qué?

Si pero no me gusta la canción, no es mi tipo de musica

• Ejercicio 9: Transformación

Usa express para transformar todo el código de `servidor.js` (el que tenía lo de opinión), nombra el archivo ahora `servidor_express.js`

Haz comentarios de qué te parece *express* en el mismo código.

Haz que se vea una foto de un árbol de tu universidad con el nombre científico del árbol en una de tus páginas html.

Escribe aquí abajo el enlace al archivo (servidor_express.js) en tu repositorio:

[LucasMejiaDiaz.github.io/servidor_express.js at main · LucasMejiaDiaz/LucasMejiaDiaz.github.io · GitHub](https://LucasMejiaDiaz.github.io/servidor_express.js)

● Ejercicio 10: Verbos

En el contexto de desarrollo web, explica los siguientes verbos GET, POST, PUT, DELETE

GET

GET se usa para pedir información al servidor. Por ejemplo, cuando el navegador entra a una página o cuando se consultan datos.

Ejemplo: pedir la lista de usuarios.

POST

POST se usa para enviar información nueva al servidor. Por ejemplo, cuando se llena un formulario o se crea un nuevo usuario.

Ejemplo: agregar un usuario nuevo.

PUT

PUT se usa para actualizar información que ya existe. Por ejemplo, cambiar el saldo o los datos de un usuario.

Ejemplo: actualizar el saldo de un usuario.

DELETE

DELETE se usa para borrar información del servidor.

Ejemplo: eliminar un usuario.

Explica los siguientes códigos, en el contexto de desarrollo web:

1xx

Son respuestas informativas. Indican que la solicitud fue recibida y que el proceso continúa.

2xx

Significan que la solicitud fue exitosa.

Ejemplo: 200 OK significa que todo salió bien.

Ejemplo: 201 Created significa que algo nuevo fue creado correctamente.

3xx

Son respuestas de redirección. Indican que el cliente debe ir a otra dirección o recurso.

4xx

Significan que hubo un error del cliente.

Ejemplo: 404 Not Found significa que la ruta o recurso solicitado no existe.

5xx

Significan que hubo un error en el servidor.

Ejemplo: 500 Internal Server Error significa que algo falló dentro del servidor.

Ejemplos usando Express relacionados con mi proyecto

Explica con un ejemplo (relacionado con tu proyecto/reto) cómo se atendería una solicitud de cada uno de esos verbos usando express. Incluye al menos una respuesta 404, 201 y 200.

```
import express from "express";
```

```
const app = express();  
const puerto = 1984;
```

```
app.use(express.json());
```

```
let usuarios = [  
  { id: 1, nombre: "Lucas", saldo: 100 },  
  { id: 2, nombre: "Andrea", saldo: 250 }  
];
```

```
// GET: consultar usuarios  
app.get("/api/usuarios", (req, res) => {  
  res.status(200).json(usuarios);  
});
```

```
// POST: crear un usuario nuevo
```

```
app.post("/api/usuarios", (req, res) => {
  const nuevoUsuario = {
    id: usuarios.length + 1,
    nombre: req.body.nombre,
    saldo: req.body.saldo
  };

  usuarios.push(nuevoUsuario);

  res.status(201).json({
    mensaje: "Usuario creado correctamente",
    usuario: nuevoUsuario
  });
});

// PUT: actualizar el saldo de un usuario
app.put("/api/usuarios/:id", (req, res) => {
  const id = Number(req.params.id);
  const usuario = usuarios.find((u) => u.id === id);

  if (!usuario) {
    return res.status(404).json({
      error: "Usuario no encontrado"
    });
  }

  usuario.saldo = req.body.saldo;

  res.status(200).json({
    mensaje: "Usuario actualizado correctamente",
    usuario: usuario
  });
});

// DELETE: eliminar un usuario
app.delete("/api/usuarios/:id", (req, res) => {
  const id = Number(req.params.id);
  const usuarioExiste = usuarios.some((u) => u.id === id);

  if (!usuarioExiste) {
    return res.status(404).json({
      error: "Usuario no encontrado"
    });
  }
});
```

```
usuarios = usuarios.filter((u) => u.id !== id);

res.status(200).json({
  mensaje: "Usuario eliminado correctamente"
});

// Ruta 404 para páginas o endpoints que no existen
app.use((req, res) => {
  res.status(404).json({
    error: "Ruta no encontrada"
  });
});

app.listen(puerto, () => {
  console.log(`Servidor escuchando en http://localhost:${puerto}`);
});
```

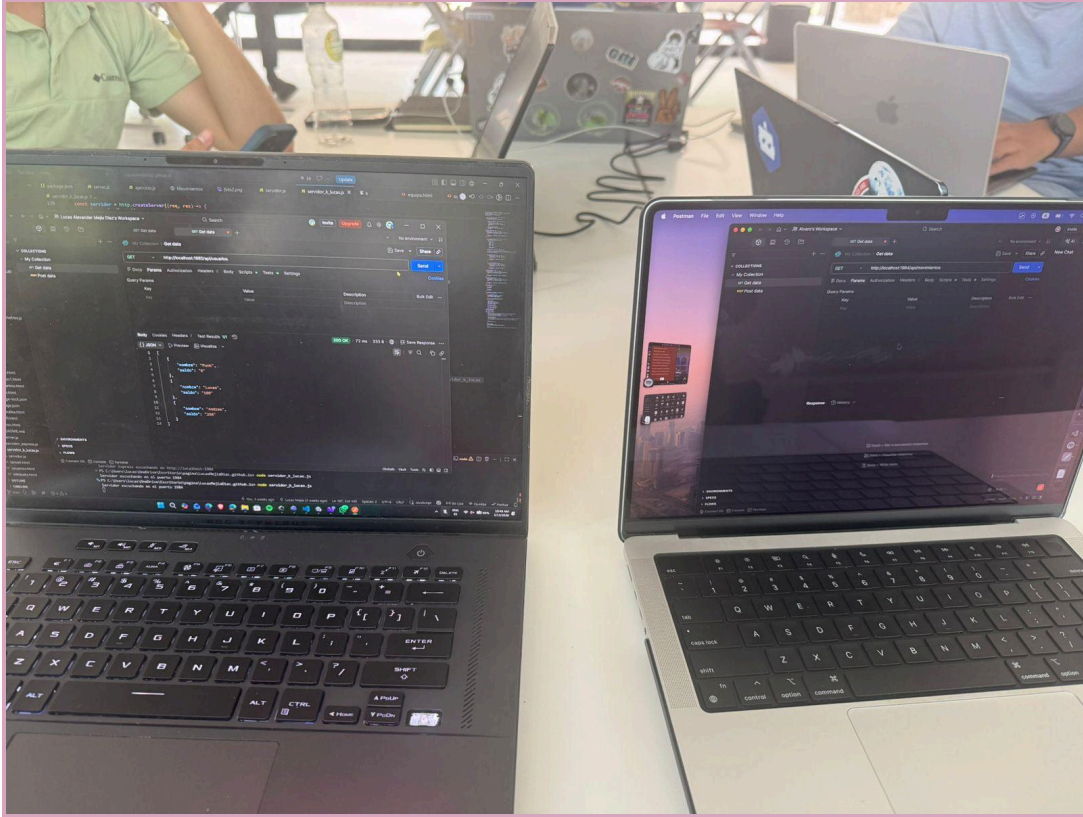
En este ejemplo:

- GET /api/usuarios responde con código 200 porque devuelve correctamente los usuarios.
- POST /api/usuarios responde con código 201 porque crea un usuario nuevo.
- PUT /api/usuarios/:id responde con código 200 si actualiza el usuario, o 404 si no lo encuentra.
- DELETE /api/usuarios/:id responde con código 200 si elimina el usuario, o 404 si el usuario no existe.

● Ejercicio 11: Postman

En equipo con tu mesa, instalen postman en cada una de sus computadoras. Tomen una misma foto en la que aparezcan todas las pantallas con postman abierto.

Foto aquí:



Explica para qué sirve Postman

Prueba lo que hiciste en el ejercicio 9 con Postman. Pon una captura de pantalla usando un GET.

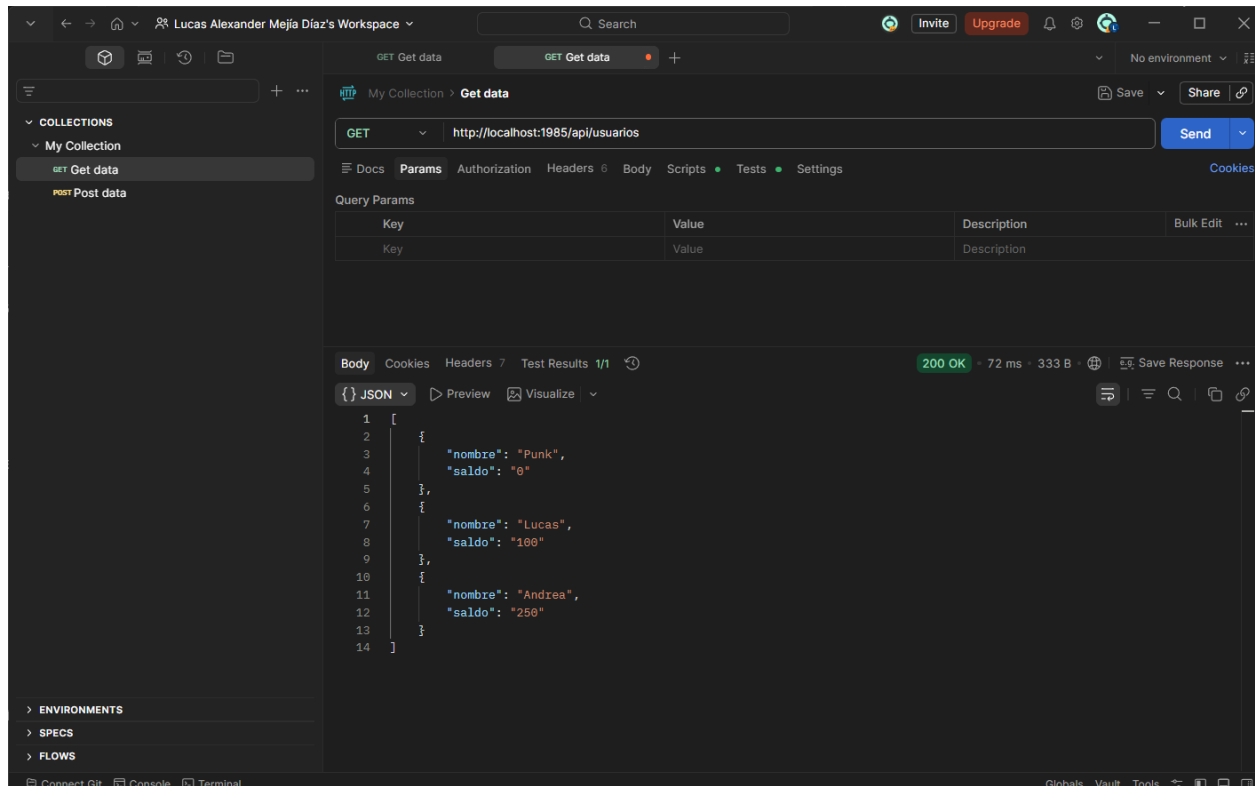
Postman sirve para probar APIs y servidores web sin necesidad de usar directamente el navegador o hacer una página completa. Con Postman podemos enviar solicitudes HTTP como GET, POST, PUT o DELETE, revisar la respuesta del servidor, ver el código de estado y comprobar si una ruta funciona correctamente.

En mi caso probé el ejercicio 9 usando una solicitud GET en Postman. Probé el endpoint:

GET `http://localhost:1985/api/usuarios`

La respuesta fue correcta porque el servidor regresó un JSON con la lista de usuarios. Esto demuestra que la ruta `/api/usuarios` de mi servidor con Express funciona correctamente.

Captura de pantalla usando GET:



• Ejercicio 12: Parámetros

Inventa un ejercicio (que esté relacionado con tu reto) para que uses parámetros en las rutas (por ejemplo /getBancos /postBancos)

Inventé un ejercicio relacionado con mi reto bancario. La idea es crear una API con Express donde se puedan consultar usuarios, consultar movimientos por tipo, hacer depósitos y eliminar usuarios usando parámetros en las rutas.

Los parámetros sirven para mandar información dentro de la URL. Por ejemplo, en esta ruta:

GET /api/usuarios/Lucas

El nombre Lucas es un parámetro. Express lo puede leer con req.params.nombre.

En mi ejercicio usé estas rutas:

GET /api/usuarios

Esta ruta muestra todos los usuarios.

GET /api/usuarios/:nombre

Esta ruta busca un usuario por nombre. Ejemplo:

GET /api/usuarios/Lucas

POST /api/usuarios/:nombre/deposito/:cantidad

Esta ruta registra un depósito usando el nombre y la cantidad como parámetros.

Ejemplo:

POST /api/usuarios/Carlos/deposito/50

DELETE /api/usuarios/:nombre

Esta ruta elimina un usuario usando su nombre como parámetro. Ejemplo:

DELETE /api/usuarios/Carlos

También agregué respuestas con códigos HTTP. Cuando encuentra un usuario responde 200, cuando crea o registra un depósito responde 201, y cuando no encuentra un usuario responde 404.

Soluciona ese ejercicio

Escribe aquí abajo el enlace al archivo en tu repositorio:

https://github.com/LucasMejiaDiaz/LucasMejiaDiaz.github.io/blob/main/ejercicio12_parametros.js

Compara ese ejercicio con el de otra persona de la clase. ¿Qué ejercicio ayuda mejor a comprender el uso de parámetros y por qué?

Comparé mi ejercicio con el de otra persona que solamente usó una ruta simple como /api/usuarios. Creo que mi ejercicio ayuda mejor a comprender el uso de parámetros porque usa varios ejemplos dentro de la URL, como :nombre, :tipo y cantidad. Eso permite ver más claramente cómo Express toma datos de la ruta y los usa para buscar, crear o eliminar información.

● Ejercicio 13: npx

Investiga con alguien más lo siguiente: npx express-generator. Importante haz un directorio diferente si quieres ejecutar el comando.

Escribe el nombre de ese alguien más:

Dario Garay

Explica lo que genera y hace ese comando.

El comando investigado fue: `npx express-generator`

Ese comando sirve para crear automáticamente la estructura inicial de un proyecto con Express. En vez de crear todos los archivos a mano, `express-generator` genera una base de proyecto con carpetas, rutas, archivos principales y configuración para empezar más rápido.

Es importante ejecutarlo en un directorio diferente porque genera varios archivos nuevos. Por ejemplo: `npx express-generator prueba_express_generator`

Después de eso, el comando indica que se debe entrar a la carpeta, instalar dependencias y correr el proyecto:

```
cd prueba_express_generator
```

```
npm install
```

```
npm start
```

Archivos y carpetas que genera:

`app.js`

Es el archivo principal de la aplicación Express. Ahí se configura Express, se agregan middlewares, se conectan las rutas y se habilita la carpeta `public` para archivos estáticos.

`package.json`

Guarda la información del proyecto, los scripts y las dependencias necesarias, como `express`, `cookie-parser`, `morgan` y `debug`.

`bin/www`

Es el archivo que levanta el servidor. Define el puerto, crea el servidor HTTP y ejecuta la aplicación.

`routes/index.js`

Contiene la ruta principal del proyecto. Normalmente responde a la ruta `/`.

`routes/users.js`

Contiene una ruta de ejemplo para usuarios. Sirve para entender cómo separar rutas en archivos diferentes.

public/

Es la carpeta donde van los archivos estáticos, como imágenes, CSS o JavaScript del navegador.

public/stylesheets/style.css

Es un archivo CSS de ejemplo para estilos.

En resumen, npx express-generator crea una plantilla básica de Express. Sirve para iniciar un proyecto más rápido y aprender una forma más ordenada de separar el servidor, las rutas y los archivos públicos.

• Ejercicio 14: Explicar

Explica a una niña o un niño de 10 años para qué sirven frameworks como Angular, React, Vue, o Svelte.

Imagina que quieres construir un carrito de juguete. Podrías fabricar cada pieza tú mismo: las ruedas, los tornillos, el volante y el motor. Sí se puede, pero tomaría muchísimo tiempo. Los frameworks como Angular, React, Vue.js o Svelte funcionan como un kit que ya trae muchas piezas listas.

Ayudan a las personas programadoras a crear páginas web y aplicaciones más rápido y de forma más organizada.

Por ejemplo, si una aplicación necesita: botones, menús, ventanas, o información que cambia en pantalla, el framework ya trae herramientas preparadas para hacer eso fácilmente.

Así, en lugar de empezar desde cero cada vez, las personas pueden enfocarse en hacer que la aplicación funcione bien y se vea mejor.

En resumen: un framework es una herramienta que facilita construir aplicaciones grandes, igual que un kit facilita construir un carrito sin fabricar todas las piezas desde cero.